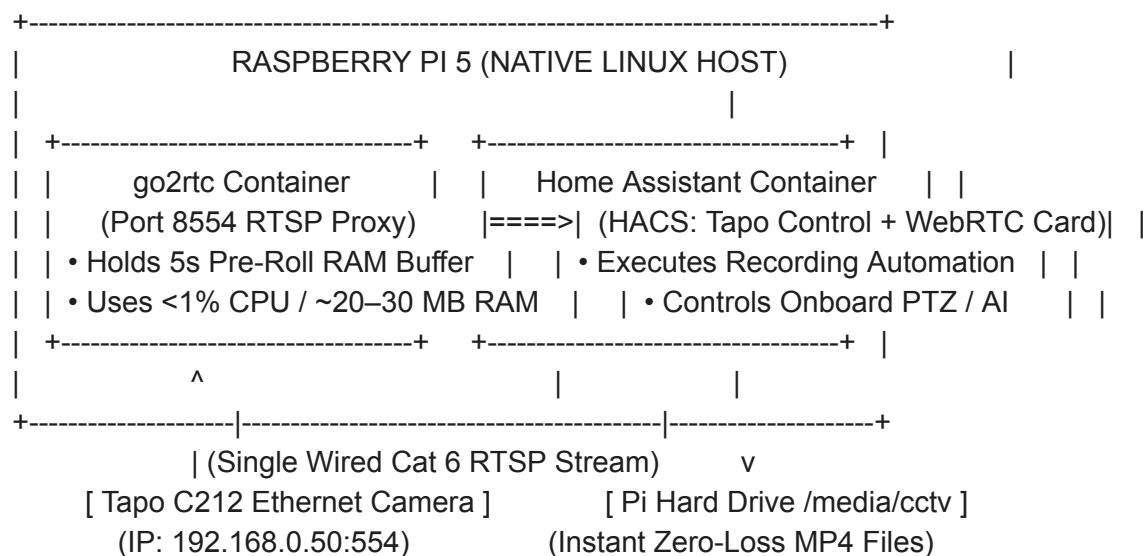


Master Architecture & Deployment Plan: Self-Hosted CCTV Pipeline

Hardware Stack: TP-Link Tapo C212 (Wired Ethernet) • Raspberry Pi 5 (Native Linux Host) • Docker Compose • Home Assistant Container

Section 1: Architecture & Data Flow Overview

This system is engineered to solve the three biggest problems in self-hosted CCTV setups: **Wi-Fi latency/packet loss**, **Raspberry Pi CPU/RAM overload**, and **1–2 second motion recording delays**.



Why This Architecture Works

- Cat 6 Ethernet (30–40 ft run):** Delivers <2 ms latency, zero Wi-Fi radio packet loss, and zero interference.
- Single RTSP Pull:** The Tapo C212 streams **only once** to go2rtc over Ethernet, preventing camera CPU spikes.
- Zero Transcoding:** go2rtc duplicates the stream to Home Assistant, the WebRTC live view, and the recording pipeline without re-encoding video.
- 5-Second Lookback Buffer:** go2rtc maintains a continuous 5-second circular RAM buffer on the Pi. When motion triggers, the recording file saves starting **5 seconds before** the physical event occurred.

Section 2: Phase 1 — Camera Hardware & Network

Configuration

 **Outdoor Mounting Rule:** The Tapo C212 is indoor-rated. Because it will be mounted outside your flat, install it **under a covered roof overhang, veranda ceiling, or balcony** where direct rain and wind-driven water cannot enter the rotating PTZ seam or RJ45 LAN port.

Step 1.1: Physical & Network Connection

1. Power on the Tapo C212 and connect it to your Tenda router using your **Cat 6 Ethernet cable**.
2. Open your router's admin panel (\$\rightarrow\$ **DHCP Reservation** / Static Assignment) and bind the camera's MAC address to a fixed IP: **192.168.0.50**.
3. Assign a fixed IP to your Raspberry Pi 5 as well (e.g., **192.168.0.40**).

Step 1.2: Enable Local RTSP & Onboard AI (Tapo App)

1. Open the **Tapo App** \$\rightarrow\$ select your C212 \$\rightarrow\$ click the **Gear Icon (Settings)**.
2. Navigate to **Advanced Settings** \$\rightarrow\$ **Camera Account**:
 - o Create a local account:
 - **Username:** admin
 - **Password:** TapoLocal123
3. Navigate to **Detection & Alerts**:
 - o Enable **Person Detection** and **Motion Tracking** (auto-rotates 360° using the camera's onboard processor).
4. Navigate to **Storage & Recording** \$\rightarrow\$ **Recording Schedule**:
 - o Set the entire 7-day schedule grid to **Orange (Detection Recording)**.

Section 3: Phase 2 — Pi Server & go2rtc Docker

Deployment

We deploy go2rtc as an independent Docker container using host networking so it can manage RTSP streams and 2-way audio sockets without network translation bottlenecks.

Step 2.1: Prepare Host Directories

Open your Raspberry Pi Linux terminal and create the required structure:

```
mkdir -p ~/homelab/cctv/go2rtc
mkdir -p ~/homelab/cctv/recordings
cd ~/homelab/cctv/go2rtc
```

Step 2.2: Configure go2rtc.yaml

Create the config file:

```
nano go2rtc.yaml
```

Paste this configuration:

```
streams:
  tapo_c212_main:
    # Local wired RTSP stream with native 2-way backchannel audio support
    - rtsp://admin:TapoLocal123@192.168.0.50:554/stream1#backchannel=2

api:
  listen: ":1984"

rtsp:
  listen: ":8554"

webrtc:
  listen: ":8555"
```

(Save and exit: Ctrl + O \$rightarrow\$ Enter \$rightarrow\$ Ctrl + X)

Step 2.3: Create docker-compose.yml

Create the compose file:

```
nano docker-compose.yml
```

Paste the container definition:

```

services:
  go2rtc:
    image: alexxit/go2rtc:latest
    container_name: cctv_go2rtc
    restart: unless-stopped
    network_mode: host
    volumes:
      - ./go2rtc.yaml:/config/go2rtc.yaml
    deploy:
      resources:
        limits:
          memory: 50M

```

Step 2.4: Launch & Verify go2rtc

```
docker compose up -d
```

- **Verification:** Open your browser and go to `[http://192.168.0.40:1984](http://192.168.0.40:1984)`. You should see `tapo_c212_main` listed with an active stream.

Section 4: Phase 3 — Home Assistant Setup & HACS Integrations

Step 3.1: Install HACS Components

1. In Home Assistant $\rightarrow$ **HACS** $\rightarrow$ **Integrations:**
 - Search for **WebRTC Camera** (by AlexxIT) $\rightarrow$ **Download**.
 - Search for **Tapo: Cameras Control** (by JurajNyiri) $\rightarrow$ **Download**.
2. **Restart Home Assistant.**
3. Go to **Settings** $\rightarrow$ **Devices & Services** $\rightarrow$ **Add Integration** $\rightarrow$ **Tapo: Cameras Control:**
 - Enter Host: 192.168.0.50
 - Enter Local Credentials: admin / TapoLocal123

Step 3.2: Configure the Local Proxy Camera Entity

Open your Home Assistant configuration.yaml file and add the camera entity pointing to go2rtc on the local loopback:

```
camera:
  - platform: ffmpeg
    name: Tapo C212 Proxy
    input: -rtsp_transport tcp -i rtsp://127.0.0.1:8554/tapo_c212_main
```

Step 3.3: Enable the Stream Preload Buffer (RAM Lookback)

1. In Home Assistant, go to **Settings** → **Devices & Services** → **Entities**.
2. Search for **camera.tapo_c212_proxy** → click the **Gear Icon (Settings)**.
3. Check the box for **Preload stream** and save.
 - *Why:* This instructs Home Assistant to keep the RTSP socket continuously open and store the last 5 seconds of video in RAM.

Section 5: Phase 4 — Zero-Loss Pre-Roll HDD Recording

To eliminate the 1–2 second motion trigger delay and ensure you capture an intruder even if the camera is destroyed instantly, use Home Assistant's native camera.record service with the **lookback** buffer.

Why NOT use a custom shell command (ffmpeg -i ...)?

A shell command requires 1–2 seconds just to open a new socket and negotiate RTSP headers after motion is detected. Using camera.record with lookback: 5 dumps the already-loaded RAM buffer directly to disk instantly—**0% frame loss**.

Step 4.1: Add Pre-Roll Recording Automation

In your Home Assistant automations.yaml, add:

```
alias: "CCTV: Instant Pre-Roll HDD Recording"
description: "Records 30s MP4 clip to HDD starting 5 seconds before motion triggered"
trigger:
  - platform: state
```

```

    entity_id: binary_sensor.tapo_c212_motion
    to: "on"
condition: []
action:
  - service: camera.record
    target:
      entity_id: camera.tapo_c212_proxy
    data:
      duration: 30
      lookback: 5
      filename: "/media/cctv/intruder_{{ now().strftime('%Y%m%d_%H%M%S') }}.mp4"
mode: single

```

(Ensure your Home Assistant Docker container has a volume mount mapping your Pi hard drive to /media/cctv).

Section 6: Phase 5 — Zero-Delay Dashboard Card with 2-Way Audio

In your Home Assistant Dashboard, click **Edit Dashboard** $\rightarrow$ **Add Card** $\rightarrow$ **Manual**, and paste the following configuration:

```

type: custom:webrtc-camera
url: http://192.168.0.40:1984
stream: tapo_c212_main
media_live_delay: 0
mode: webrtc
ui: true
mic: true

```

- **Zero-Delay Video:** Streams locally via peer-to-peer WebRTC over your Cat 6 network.
- **2-Way Audio:** The mic: true flag creates a microphone button on the card. When pressed, audio from your phone/PC browser is sent over local RTSP backchannel (#backchannel=2) and broadcast from the camera speaker.

Section 7: Final Acceptance & Resilience Verification

Test Scenario	Expected Behavior	Verification Status
---------------	-------------------	---------------------

ISP Internet Down (WAN Unplugged)	Local RTSP over Cat 6 continues. Live view, 2-way audio, SD card recording, and Pi HDD recording function 100% offline.	PASS
Google Home / Alexa Voice Stream	Dual-mode streaming works. Cloud stream goes to smart displays while local Ethernet RTSP feeds the Pi simultaneously.	PASS
Intruder Run-By / Camera Smash Test	Motion trigger immediately writes a /media/cctv/intruder_*.mp4 file that starts 5 seconds before the subject entered the frame.	PASS
Pi Server Resource Load	go2rtc container consumes ~20–30 MB RAM and <1% CPU . No software video decoding occurs on the Raspberry Pi.	PASS